Jr.Pen 2.3. Dive Into Pentesting

Task 1 - Introduction

Leveraging foundational penetration testing to proactively uncover vulnerabilities and communicate risk in complex environments. Introducing foundational penetration testing concepts, ethical principles, risk relationships, and the security mindset.

Learning Objectives

- Distinguish between penetration testing and malicious hacking based on authorisation, scope, coverage, and responsibility.
- Describe the core focus areas of web application and network penetration testing.
- Define vulnerability, threat, and risk, and apply the `Vulnerability x Threat = Risk` relationship to evaluate real-world scenarios.
- Outline the four stages of the risk management cycle, and recognise when risks are mitigated, accepted, or transferred.
- Identify the common reasons why vulnerabilities exist, including human assumptions, software bugs, system complexity, over customisation, and technical or design flaws.
- Apply the mindset and habits of an effective penetration tester, such as attention to detail, contextual thinking, and avoiding tool over-reliance or tunnel vision.
- Follow best practices during an engagement, including maintaining good notes, collecting evidence, managing time, communicating proactively, and staying professional.
- Uphold the principles of ethics, permission, and trust throughout an engagement to protect both the organisation and the tester.

Task 2 - Penetration Testing vs. Malicious Hacking

Understanding penetration testing as an authorized security assessment that identifies weaknesses to help organizations reduce risk. Highlighting the critical benefits of penetration testing in prioritizing risk, ensuring compliance, and validating security controls.

Penetration Testing vs Malicious Hacking

Core Factor	Penetration Tester	Malicious Attacker
Authorisation	Penetration testers operate with explicit written consent from the system owner.	Attackers operate without consent and violate the rights and security of the organisation.
Scope	Penetration testers operate within a clearly defined scope that is set by the organisation in order to protect critical systems.	Attackers are not restricted by scope and will target anything that helps them achieve their goal.
Coverage	Penetration testers aim for broad coverage and assess multiple areas of a system.	Attackers focus on the quickest or most effective path to success, and often target systems and data that can provide financial benefit.
Responsibility	Penetration testers are accountable for their actions and the impact of their work, and are expected to act professionally at all times.	Attackers do not feel accountable for their actions and do not take responsibility for the damage that they cause.

Answer the questions below

> What is the common shortened term for penetration testing?

pentesting

> Which actor aim for broad coverage and assesses multiple areas of a system?

Penetration Tester

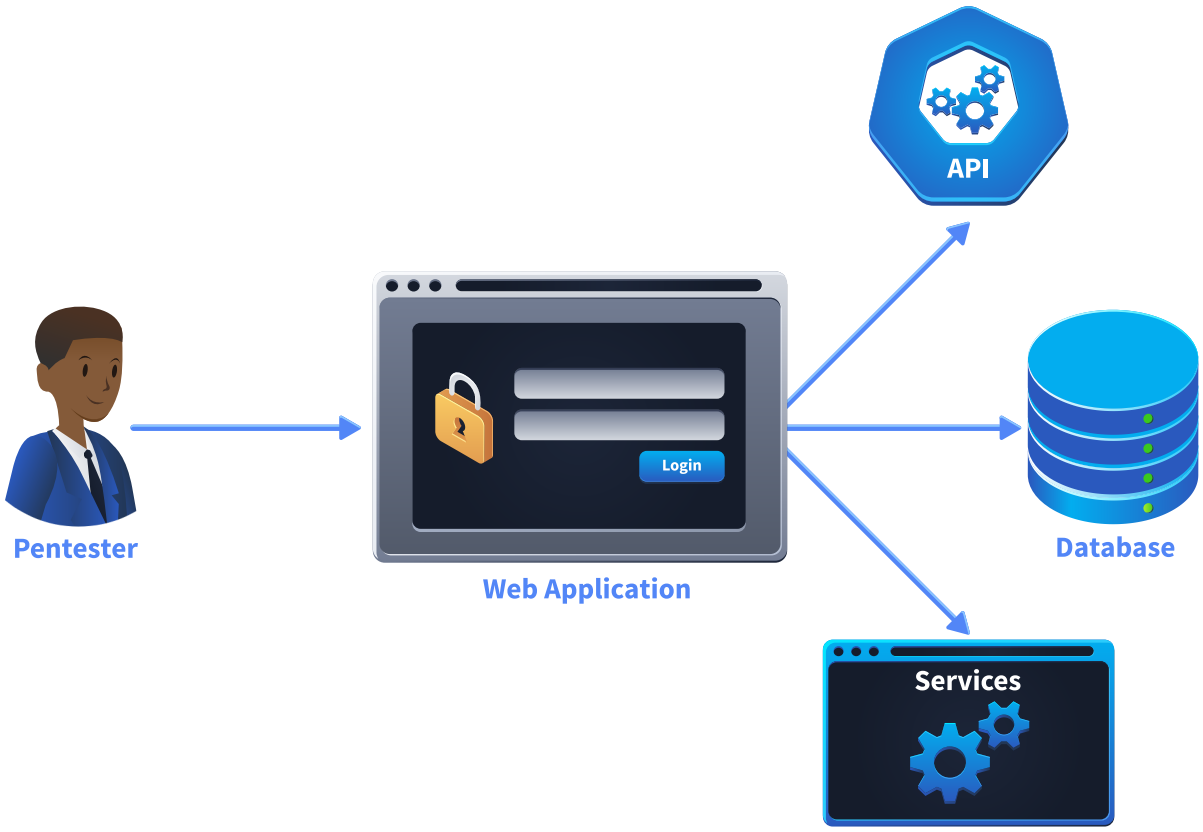
> Which actor focuses on the quickest path to success?

Attacker

Task 3 - Penetration Testing Focus Areas

In a penetration test, coverage is critical in order to fully assess the overall security posture of an organisation. To achieve meaningful coverage, it is important to understand the different areas that a penetration tester looks at. This task will cover areas that are assessed when performing web application and network penetration testing.

Web Application Penetration Testing: Web application penetration testing evaluates an internet-exposed application's security by simulating user interactions and API calls to identify vulnerabilities in areas like authentication, authorization, and data processing.



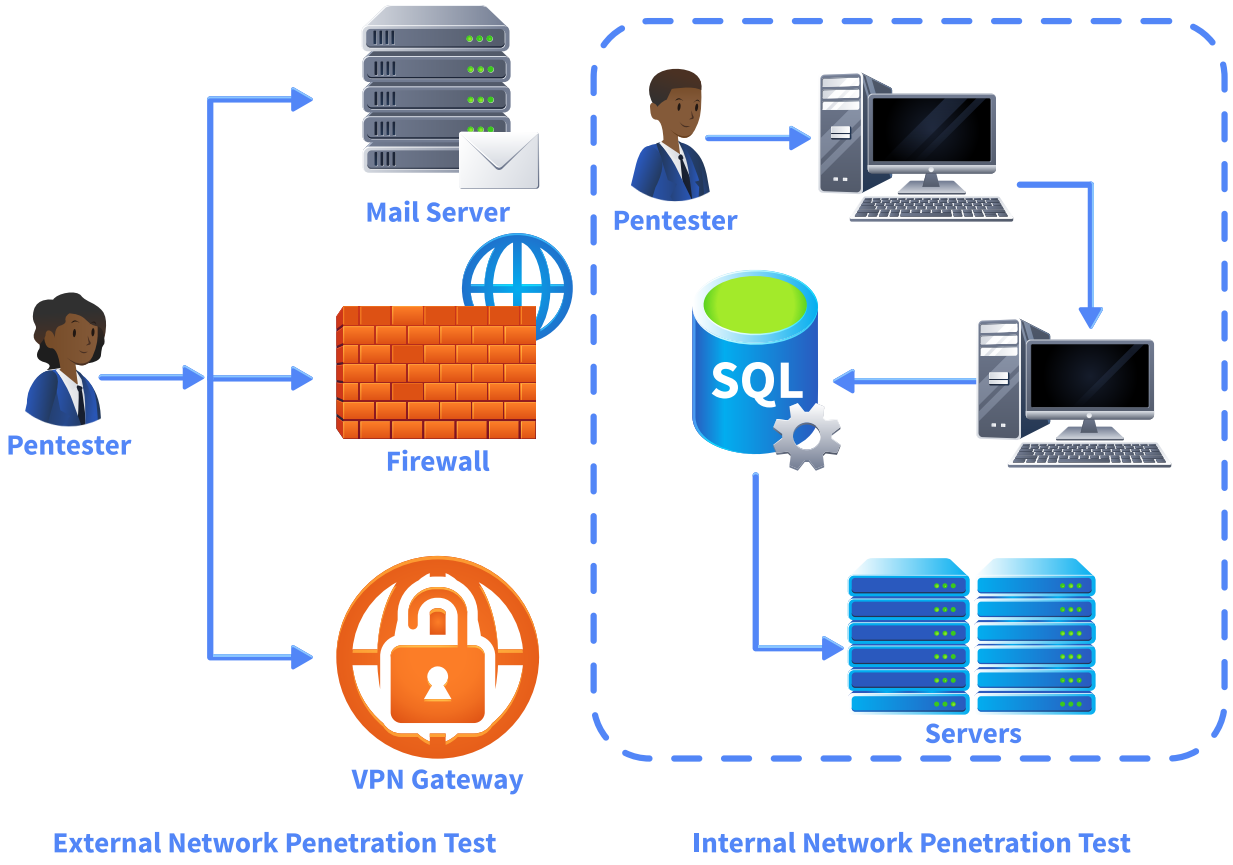
The diagram above outlines a simple interaction between a penetration tester and a web application and its components.

- **Authentication**: This area is evaluated for weaknesses in the application's credential handling, password policy, account lockout mechanisms, multi-

factor authentication (MFA) implementation, protection against automated attacks such as brute-force and credential stuffing, password-reset flow and overall authentication-related logics.

- **Authorisation**: This area is evaluated for weaknesses in the application's access control mechanism, ensuring users can only access resources and perform actions permitted by their role, and protection against vertical and horizontal privilege escalation attacks.
- **Session management**: This area is evaluated for weaknesses in how sessions are created, maintained, and invalidated. This includes session fixation risks, session invalidation after logout, idle timeout enforcement, secure cookie attributes, and protection against cross-site request forgery (CSRF).
- **Input and output validation**: This area is evaluated for weaknesses in the application's data handling controls, including protection against injection attacks, data type validation, and output handling.
- **Security configuration**: This area is evaluated for gaps in the server and application configurations, including security headers, error handling behaviour, rate-limiting controls, cryptographic configuration, and exposure of unnecessary services or features.

Network Penetration Testing: Evaluating internet-facing infrastructure and security controls from an external perspective to protect against unauthorized access. Assessing internal security controls, network segmentation, and trust relationships to limit the impact of a compromise in an assumed breach scenario.



The diagram above shows a simple interaction between a penetration tester and a network, both from an external and internal perspective.

- **Authentication mechanisms**: This area is evaluated for weaknesses in network-level authentication controls such as password policy, multi-factor authentication (MFA) enforcement, credential reuse, use of default credentials, and protection against attacks on services such as admin portals, VPN, SSH, or RDP.
- **Authorisation and access controls**: This area is evaluated for weaknesses in network access control mechanisms, ensuring users and systems can only access resources permitted by their role or trust level.
- **Network segmentation and trust relationships**: This area is evaluated for weaknesses in user and system trust relationships, firewall rules, and isolation controls.

- **Configuration and patch management**: This area is evaluated for gaps in device configurations and services, such as use of outdated software, insecure default configuration, and weak encryption protocols.

Identifying the attack surface uncovers potential entry points and defines the scope for penetration testing.

Answer the questions below

> What type of network penetration test focuses on internet-facing infrastructure from the perspective of an unauthorised user?

External

> During testing, you discovered that session cookies remain valid after a user logs out of the application. Which testing focus area does this issue fall under?

Session Management

Task 4 - Vulnerability, Threat, and Risk

Vulnerability, threat, and risk are core concepts in cyber security that are commonly used together. They allow organisations to assess and manage their security concerns in a structured way.

Vulnerability: An exploitable weakness or gap in an environment that creates an opportunity for security compromise.

Threat: Any malicious or non-malicious source of danger capable of exploiting a vulnerability to cause harm.

Risk: The potential damage determined by combining the impact and likelihood of a threat successfully exploiting a vulnerability.

Low Risk: Imagine a web application displays detailed error messages when invalid input is submitted.

- **Vulnerability:** Verbose error messages revealing internal file paths
- **Threat:** Attackers triggering errors to gather system information
- **Risk:** Limited information disclosure that may help reconnaissance but does not directly compromise the system

High Risk: Imagine a web application that allows users to change an account ID parameter in a request and access other users' data.

- **Vulnerability:** Broken access control allowing unauthorised access to user data
- **Threat:** Attackers modifying request parameters to retrieve or alter other sensitive user information
- **Risk:** Exposure or manipulation of sensitive customer data, leading to data privacy violations and regulatory consequences

Risk Management: Identifying, evaluating, and controlling security risks to reduce or eliminate their impact and likelihood. Risk management is typically an ongoing cycle consisting of four stages:

- **Identification**: Identifying assets, vulnerabilities, and potential threats that could affect the organisation.
- **Analysis**: Evaluating risks to determine the potential impact and likelihood of exploitation, which translates to the severity of each risk and helps plan remediation efforts.
- **Mitigation**: Reducing or controlling identified risks. This may include applying security patches, strengthening access controls, improving configurations, implementing monitoring tools, or redesigning insecure processes.
- **Monitoring**: Continuous monitoring of risks to ensure that controls remain effective, new vulnerabilities are detected, and emerging threats are addressed promptly.

Organizations manage unmitigated risks by accepting low-impact, costly-to-fix risks or transferring residual financial liabilities through mechanisms like cyber insurance.

Answer the questions below

> An organisation patched a high-severity issue that you reported. What stage of the risk management cycle does this activity fall under?

Mitigation

> Would an SQL-injection vulnerability present a higher risk on an external-facing application or an internal-facing application?

External-facing application

Task 5 - Why Vulnerabilities Exist

Vulnerabilities can occur through flaws, features, or human error, which can result in compromise of data and resources, and attackers can exploit one or more to achieve their goal.

Human Assumptions: Developers assuming users will behave as intended leads to omitted controls that attackers can exploit through non-standard interactions.

Software Bugs: Coding errors and inadequate input validation create unintended behaviors that allow attackers to manipulate application logic and database queries.

System Complexity: Interconnected modern architectures increase the likelihood of overlooked misconfigurations that expose sensitive endpoints and APIs.

Over Customisation: Tailored features often bypass standard security practices and become difficult to maintain, patch, and keep secure over time.

Technical and Design Flaws: Omitting security requirements from the initial architecture introduces foundational logic flaws that undermine protective mechanisms.

Common Causes and Resulting Vulnerabilities

Root Cause	Example Scenario	Resulting Vulnerability
Human Assumptions	Developer assumes users only upload images to a profile photo field, no validation is in place.	Unrestricted file upload: An attacker uploads a web shell and executes OS commands on the server.
Software Bugs	Form data is concatenated into a database query rather than handled through parameterised inputs.	SQL injection: An attacker crafts input to extract or modify sensitive records from the database.
System Complexity	Multiple integrated services make it easy to overlook a	Exposed admin API: An attacker reaches admin functions that were

Root Cause	Example Scenario	Resulting Vulnerability
	misconfigured API endpoint.	never meant to be publicly accessible.
Over-Customisation	Custom authentication replaces a standard framework with logic that is hard to maintain.	Weak authentication: Outdated hashing and broken session logic expose user accounts to takeover.
Technical and Design Flaws	Authenticated session is issued before the MFA step is completed.	MFA bypass: An attacker navigates to authenticated pages without completing the MFA process.

Answer the questions below

> A developer implemented an "Upload Resume" feature in a career portal without implementing guardrails. What is the reason that would cause an unrestricted file-upload vulnerability?

Human Assumptionsdobre

Task 6 - The Pentester Mindset

Good vs Bad Mindset During a Penetration Test

A good mindset when performing penetration tests includes several qualities. The following examples highlight characteristics that expand on these qualities:

- **Understanding the system**: In most cases, systems behave differently from one another. Having a strong understanding of how the target operates will help testers plan ahead and gain the context to explore potential exploitation opportunities.
- **Attention to detail**: Observing how the application behaves and noticing small differences in certain scenarios often results in impactful findings.
- **Constant curiosity**: Constantly asking "What if?" type of questions helps uncover many ideas that can be explored and open further opportunities to identify weaknesses.
- **Prioritising critical areas**: Focusing on high-impact functions before lower-risk features ensures that effort is directed towards areas where security issues would cause greater business impact.

- **Thinking in context**: Understanding how a feature is used and what data it handles helps assess risk more accurately and allows findings to be evaluated based on real-world consequences rather than technical severity alone.
- **Creative thinking**: Hardened systems usually create an assumption that weaknesses may not exist. Approaching these systems with creativity allows a tester to identify weaknesses that could otherwise be overlooked. This approach may involve linking two vulnerabilities in order to achieve greater impact.

A poor mindset when performing penetration tests can reduce the quality of an assessment and lead to missed or inaccurate findings. The following examples highlight characteristics that could result in unreliable testing:

- **Rushing to exploitation:** Attacking a system without understanding how it works often leads to overlooked weaknesses. Additionally, this approach can increase the risk of disrupting systems, which could lead to limitations in testing.
- **Ignoring context:** Focusing only on technical success without considering business impact could result in findings that are lacking real-world relevance.
- **Over-reliance on tools:** Depending too much on automated tools without proper analysis may produce false positives or miss logic-based issues that require manual testing.
- **Making assumptions:** Assuming how a system or functionality works without verification may cause testers to overlook important behaviours.
- **Tunnel vision:** It is easy to become fixated on one functionality, especially if a tester thinks that it will lead to a critical finding. This behaviour typically leads to time wasted, which can reduce coverage and prevent exploration of other areas that may contain more significant weaknesses.
- **Blindly following a checklist**: While having a comprehensive checklist is useful in ensuring full coverage, depending on it too much could reduce a tester's ability to think creatively and lead to missed opportunities for more significant findings. Additionally, it could create an assumption that testing is finished once the checklist is completed.

The table below summarises the contrast between effective and ineffective mindsets during a penetration test.

Effective Mindset	Ineffective Mindset
Understand the system: Study how the target operates before testing	Rushing to exploitation: Attacking without understanding the system
Attention to detail: Notice subtle behavioural differences	Ignoring context: Reporting findings without business relevance
Constant curiosity: Ask "what if?" to uncover new opportunities	Over-reliance on tools: Missing logic flaws that require manual testing
Prioritise critical areas: Focus effort on high-impact functions first	Making assumptions: Guessing system behaviour without verification
Think in context: Assess risk by real-world consequence	Tunnel vision: Fixating on one area at the cost of coverage
Creative thinking: Chain vulnerabilities for greater impact	Blindly following a checklist: Missing deeper issues outside the list

Common Best Practices During a Penetration Test

Maintaining Good Notes: Documenting activities and findings ensures reproducibility and prevents losing track of critical issues throughout the assessment.

Collecting Evidence: Capturing screenshots, logs, and outputs validates findings, demonstrates impact, and saves time during reporting.

Managing Time: Prioritizing high-impact areas and drafting report contents as testing progresses prevents project delays and burnout within fixed engagement timelines.

Proactive Communication: Regularly reporting progress and early blockers minimizes coverage impact and allows quick operational adjustments.

Staying Professional: Respecting scope, safeguarding sensitive data, and communicating findings through their business impact builds trust and maintains tester credibility.

Answer the questions below

> What characteristic includes attacking without understanding how a functionality or system works?

Rushing to exploitation

> What common best practice helps in reproducing findings later?

Maintaining good notes

> What common best practice could help prevent blockers from impacting the coverage of a penetration test?

Proactive communication

Task 7 - Ethics, Permission, and Trust

Ethics, Permission, and Trust (Overview): Upholding ethical decisions, explicit authorization, and mutual trust ensures penetration testing remains a controlled, protective, and professionally credible practice.

Ethics: Acting responsibly and intentionally when interacting with sensitive systems and critical organization assets. Examples of how ethical behaviour is enforced in practice during a penetration test are:

- Respecting the defined scope and avoiding activities outside authorised boundaries
- Avoiding actions that could disrupt systems and services
- Handling sensitive data responsibly and only accessing data that is required to demonstrate impact
- Stopping and reporting unexpected access to highly sensitive or out-of-scope systems
- Redacting sensitive data in reports to prevent unintended disclosure

Permission: Securing formal authorization and a defined scope prior to testing to ensure alignment with organizational expectations. The following examples illustrate how permission is respected in practice during a penetration test:

- Obtaining written authorisation before initiating any testing activities

- Defining a clear scope and adhering to the agreed-upon scope of testing
- Confirming testing windows and approved methods before conducting potentially disruptive actions
- Seeking clarification when uncertainty exists about whether a system or action is in scope
- Pausing and notifying the organisation if activities may exceed authorised boundaries

Trust: Building confidence between external testers and organizations to safely execute independent evaluations. The examples below illustrate how trust can be built and maintained during a penetration test:

- Providing status updates periodically to reassure stakeholders that testing is progressing as planned
- Being transparent about limitations, blockers, and uncertainties during testing in order to receive proper assistance
- Reporting findings accurately and clearly demonstrating business impact
- Providing actionable recommendations that help eliminate or mitigate risks
- Addressing organisational concerns promptly and adapting communication style to technical and non-technical stakeholders

Maintaining these three pillars safeguards both the tester and target environment while preserving the integrity and reliability of security testing.

Answer the questions below

> What defines boundaries during a penetration test?

Scope

> What type of impact should findings demonstrate clearly?

Business impact

> What type of data must be removed from reports to prevent unintentional disclosure?

Sensitive data

Task 8 - Knowledge Recap

Effective penetration testing relies on combining technical skills with critical thinking, business context, clear evidence of impact, and strong professional ethics to deliver meaningful security improvements and build trust.

Practical Assessment: Your task is to help a tester determine the best decision that they can make to maintain professionalism during an assessment. Upon completion of the task, paste the presented flag as the answer to the question below.

Answer the questions below

> Complete the task and submit the flag.

THM{L3t$_d1v3_1nt0_Pen7es71ng!}

1. A tester is scheduled to begin a penetration test today. The client has not yet provided written authorisation confirming permission to test. What should the tester do?

Perform passive reconnaissance only until approval arrives.
Follow up with the client but begin testing anyway to save time.
* Wait until written authorisation is received before performing any testing.
Start testing but avoid exploitation.

2. Before launching scans, the tester notices an additional domain belonging to the client that is not listed in the scope document. What is the best action?

Ignore documentation and test all discovered assets.
Test the domain since it belongs to the same organisation.
Perform limited enumeration to confirm risk exposure.
* Request clarification from the client before interacting with the domain.

3. The tester plans to run automated vulnerability scans against a production web application during business hours. What should the tester do first?

Run scans quietly without notifying anyone.
* Confirm approved testing windows and scan intensity with the client.

Run the scan using default aggressive settings.
Reduce scan speed slightly and proceed immediately.

4. During testing, the tester gains administrative access to the application and can view customer data. What is the best decision?

Explore all available data to identify additional risks.
Download the full database to demonstrate impact.
* Capture minimal evidence required to prove admin access and stop further interaction.
Modify records to demonstrate full control.

5. While attempting exploitation, the tester realises that continuing the attack could cause the application to crash or disrupt business operations. What should the tester do?

Continue exploitation carefully to complete validation.
Attempt the exploit quickly to minimise potential disruption.
Proceed with exploitation outside peak business hours without informing anyone.
* Pause testing and seek client approval before continuing.

6. The tester discovers credentials that appear to grant access to another internal system not included in scope. What is the most appropriate action?

Perform read-only access testing.
* Document the finding and notify the client without accessing the system.
Test briefly and exclude results from the report.
Attempt login to confirm whether access works.

7. While preparing the report, the tester notices one vulnerability could not be fully validated due to time constraints. What should the tester do?

Increase severity to encourage remediation.
Report it as confirmed to ensure visibility.
* Document it clearly as partially validated with noted limitations.
Exclude it entirely from the report.

8. After delivering the final report, the client asks the tester to clarify several findings and provide guidance on remediation steps. What is the best action for the tester?

Share only the report again without further explanation.
Inform the client that the engagement has ended and additional support requires a new contract.
Direct the client to publicly available security documentation instead.
* Provide clarification and assist the client in understanding remediation recommendations.